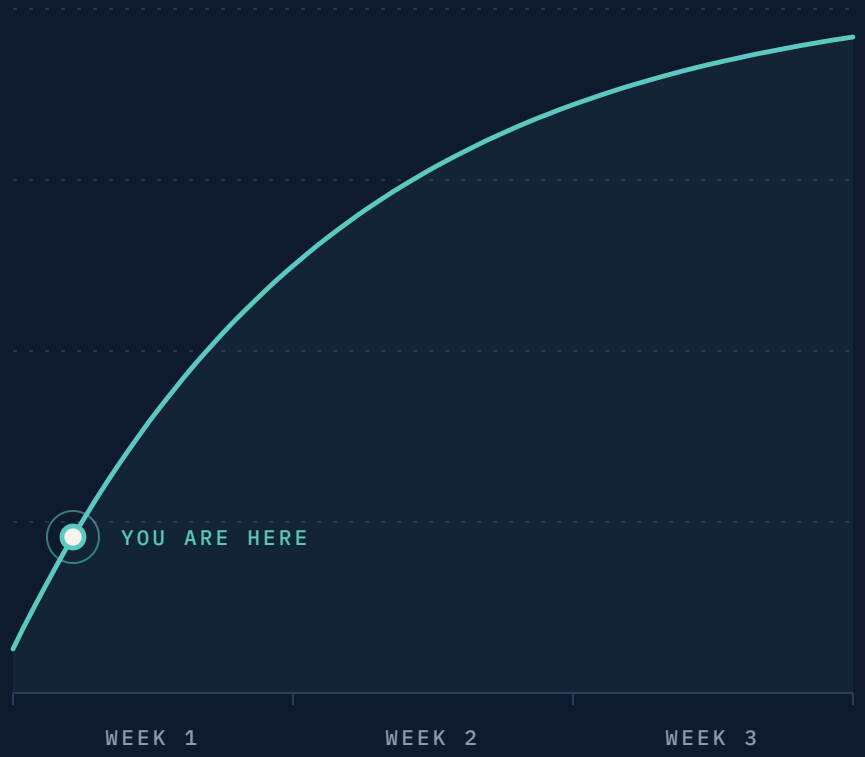
WEEK 1 · DAY 2 · OVERVIEW

Cash Flows and Price From Yield



Created by **Jeff Lenamon**

ABOUT 40 MIN WITH EXERCISES / 7 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

Notebook 01 valued each bond's 100 at maturity. That is not a price: Quorvane also pays ten coupons. List every payment each bond makes, price all four from their yields, and split each price into the 100 and the coupons. Put both steps in bondmath, docstring first, and prove them against Excel's PV and PRICE. Tell me why clean and dirty are the same number today. Read the change before you save it, and take it home.

ABOUT 40 MIN WITH EXERCISES / 7 SECTIONS / 5 EXERCISES

Your Route Through Day 2



FIGURE 1 Seven sections, in order, with about how long each takes.

Time. About 27 minutes for the lesson, then about 15 for the five exercises.

2.1 The Cash Flows of a Bullet Bond

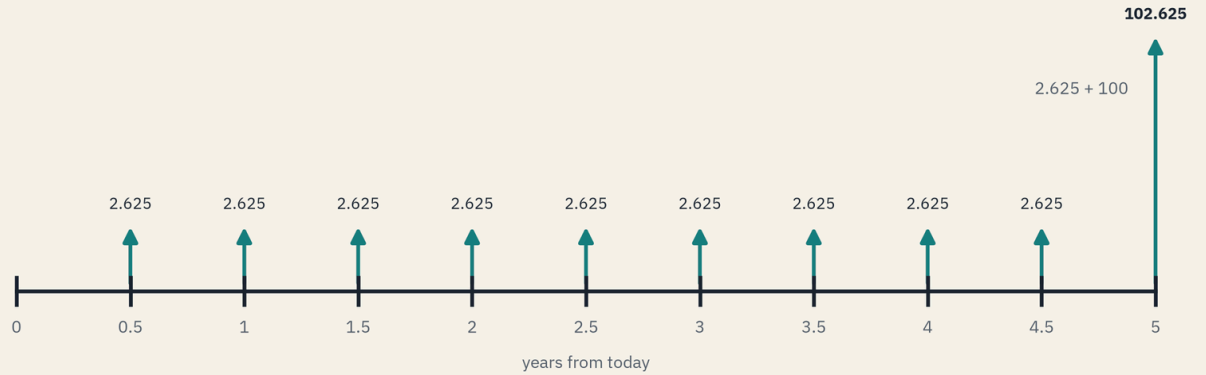


FIGURE 2 Quorvane pays nine coupons of 2.625 and a tenth payment of 102.625 with the 100, one every half-year.

FOUR BONDS' PAYMENTS

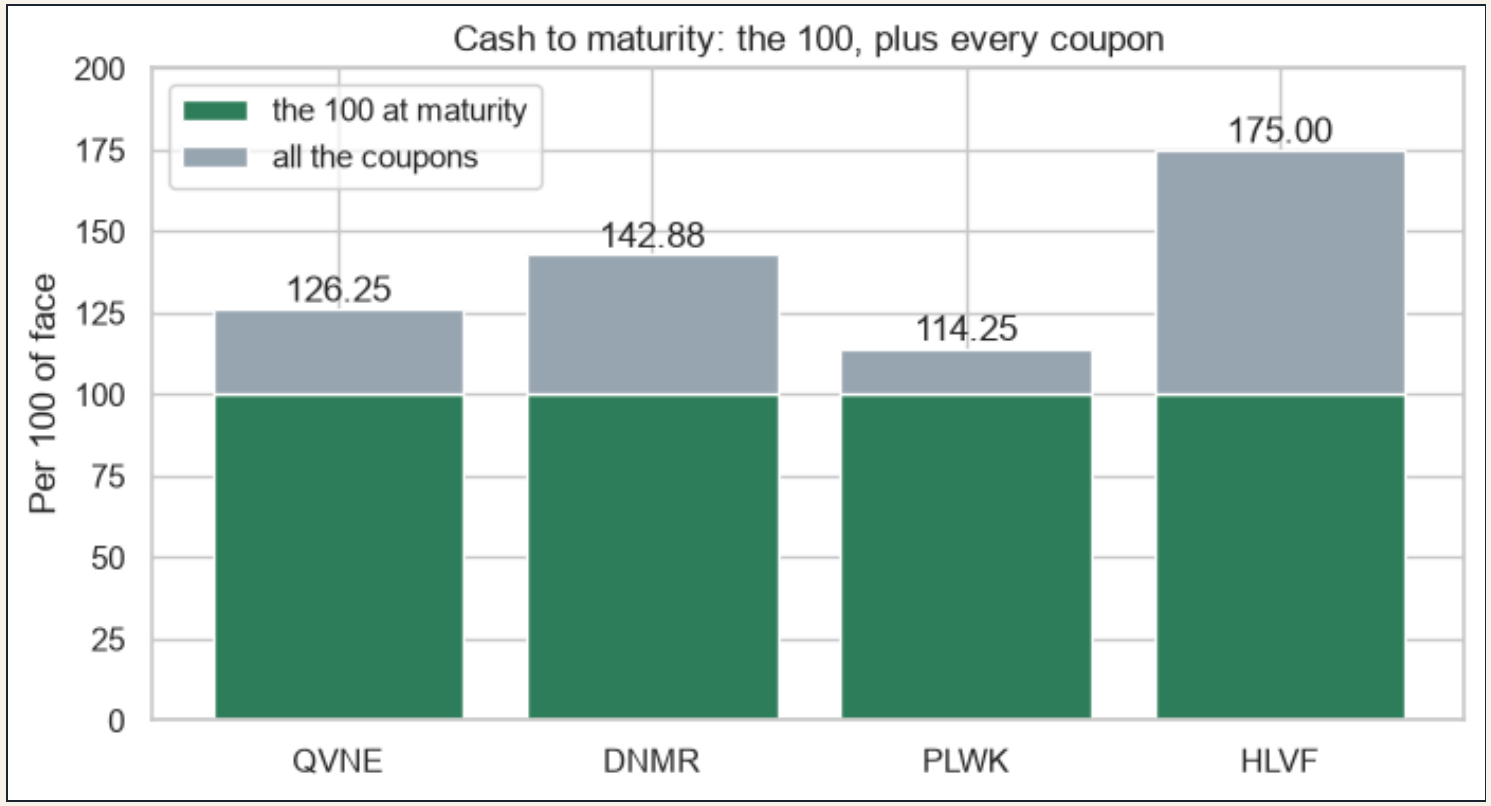
TICKER	PAYMENTS	TOTAL_CASH
QVNE	10	126.25
DNMR	14	142.88
PLWK	6	114.25
HLVF	20	175.00

ALL THE CASH

```
Total cash: 126.25 per 100 of face, 26.25 of it coupons
```

Halvering pays 175.00 per 100 and Pellwick 114.25; what each is worth today is the next question.

Cash to Maturity, Stacked



EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
The payments	`=5.25/2` in B1, filled to B9; `=5.25/2+100` in B10	2.625, then 102.625
All the cash	`=SUM(B1:B10)`	126.25

Halvering pays 175.00 per 100 and Pellwick 114.25; what each is worth today is the next question.

2.2 Price as the Sum of Discounted Cash Flows

THE CODE

```
period_yield = 5.4 / 100 / 2
quorvane_flows['discount_factor'] = 1 / (1 + period_yield) ** quorvane_flows['payment']
quorvane_flows['present_value'] = quorvane_flows['cash_flow'] * quorvane_flows['discount_factor']
quorvane_flows.round(6)
```

EACH PAYMENT, DISCOUNTED

	CASH_FLOW	DISCOUNT_FACTOR	PRESENT_VALUE
0	2.625	0.973710	2.555988
1	2.625	0.948111	2.488791
2	2.625	0.923185	2.423360
3	2.625	0.898914	2.359650
4	2.625	0.875282	2.297614

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Each payment discounted	1 to 10 in A1:A10; `=B1/(1+5.4/100/2)^A1` in C1, filled to C10	2.555988 to 78.622841

Payment k waits k half-years, so its discount factor is 1 / 1.027 to the power k, notebook 01's factor for its own wait.

2.3 cash_flows and price_from_yield in Your Module

APPEND CASH_FLOWS

```
def cash_flows(coupon_pct: float, years: float, frequency: int = 2) -> list[tuple[float, float]]:
    periods = years * frequency
    coupon = coupon_pct / frequency
    for k in range(1, periods + 1):
        amount = coupon + 100 if k == periods else coupon
    return flows
```

APPEND PRICE_FROM_YIELD

```
def price_from_yield(coupon_pct: float, yield_pct: float, years: float, frequency: int = 2) -> float:
    period_yield = yield_pct / 100 / frequency
    price = 0.0
    for time_years, amount in cash_flows(coupon_pct, years, frequency):
        price += amount / (1 + period_yield) ** (time_years * frequency)
    return price
```

The coupon rule lives in one place: `price_from_yield` never builds a payment itself.

2.4 Excel's Prices as Tests

A HAND CASE

```
%%writefile -a test_bondmath.py
REFERENCE = read_rows("bondmath_excel_reference.csv")
def test_cash_flows_count_and_final_flow():
    flows = bondmath.cash_flows(6.0, 5, 2)
    assert len(flows) == 10
    assert flows[0] == pytest.approx((0.5, 3.0))
    assert flows[-1] == pytest.approx((5.0, 103.0))
def test_cash_flows_raises_on_part_periods():
    try:
        bondmath.cash_flows(6.0, 2.3, 2)
    except ValueError:
        return
```

A TEST WITH A FILTER

```
def test_price_from_yield_matches_excel():
    coupon_date_rows = [row for row in REFERENCE if row["coupdaybs"] == "0"]
    assert coupon_date_rows, "no coupon-date rows in the reference file"
    for row in coupon_date_rows:
```

HOW MANY EXCEL NUMBERS

```
20 -PV rows, 1.0 to 30.0 years. By frequency: {1: 1, 2: 17, 4: 2}
17 PRICE rows on a coupon date. By basis: {0: 13, 1: 4}
```

`6 passed`: `price_from_yield` **matches Excel's -PV on 20 rows and PRICE on the 17 that settle on a coupon date; 2.5 says why only those.**

2.5 Clean Equals Dirty on a Coupon Date

ON A COUPON DATE

```
Largest accrued on the 17 rows: 0.000000 per 100
```

BETWEEN COUPONS

```
99080CAE8: last coupon 2026-06-18, 102 of 180 days ago
Clean 106.726595 + accrued 1.983333 = dirty 108.709928
```

ACCRUED, CLEAN, DIRTY

Accrued interest is the part of the next coupon earned since the last one; the buyer pays it to the seller.

The **clean price** leaves it out: the quoted price, and what Excel's `PRICE` returns.

The **dirty price** is clean plus accrued: the cash that changes hands.

On a coupon date accrued is zero, so one number is both prices; between coupons they part, and notebook 08 prices that day.

See every line that changed before you commit.

2.6 Read the Change, Then Commit

LINE BY LINE

```
!git -C "{work}" diff bondmath.py
```

```
diff --git a/bondmath.py b/bondmath.py
index 8590487..5a4e6a1 100644
--- a/bondmath.py
+++ b/bondmath.py
```

COMMIT AND READ THE HISTORY

```
!git -C "{work}" commit -m "Notebook 02: cash_flows and price_from_yield"
!git -C "{work}" log --oneline
```

```
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

READ IT FIRST

Read a change before you save it, whether it is yours, a colleague's, or an AI's.

Added lines only, nothing of notebook 01 touched: a change worth committing.

Bring It Home

IN YOUR OWN FOLDER

```
# the work folder the setup cell printed
$work = "$env:TEMP\pfi_course2_02_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
python -m pytest -q
git diff --stat
git add bondmath.py test_bondmath.py
git commit -m "Notebook 02: cash_flows and price_from_yield"
git log --oneline
```

my_bondmath **holds four functions, six tests, and two commits.**

You Can Now

- List a bullet bond's payments: the coupon split by the frequency, and the 100 with the last.
- Price a bond on a coupon date as the sum of its discounted payments, and split the price into the 100 and the coupons.
- Get the same price in Excel four ways: a discounted column, **`NPV`**, **`-PV`** with the coupon, and **`PRICE`**.
- Write a function that returns a list of tuples, and one that calls another.
- Test against Excel with a filter that must find rows.
- Say why clean equals dirty on a coupon date.
- Read a change with **`git diff`** before you commit it.

NEXT

WEEK 1 · DAY 3

The Price-Yield Relationship

Why a bond prices above, at, or below 100, how its price moves with its yield and with time, and a yield restated at another frequency, in a new function tested against Excel's EFFECT and NOMINAL.

